



Laboratório 7 - Programação 6

Uso Acadêmico Exclusivo

Aluno: Djordan Gabriel Bezerra Moura

Otimização de Banco de Dados e Uso Avançado do Redis no BattleTanks

1. Introdução

Este laboratório aplica técnicas de otimização de consulta com PostgreSQL e Entity Framework Core, e amplia o uso do Redis no projeto capstone **BattleTanks Multiplayer**, com o objetivo de sustentar várias partidas simultâneas.

O ponto de partida é o esquema que já existe na branch **LB-6**: três entidades — **Jogador**, **GameSession** e **Pontuacao** — mapeadas por **BattleTanksContext**, e um **RedisStateService** que hoje guarda apenas estado efêmero de partida (paredes destruídas e os últimos 100 eventos).

A premissa que orienta as decisões é que **otimização sem medição é chute**. Cada mudança proposta abaixo vem acompanhada da razão pela qual ela deve ajudar, e do comando que confirma se ajudou.

2. Diagnóstico do estado atual

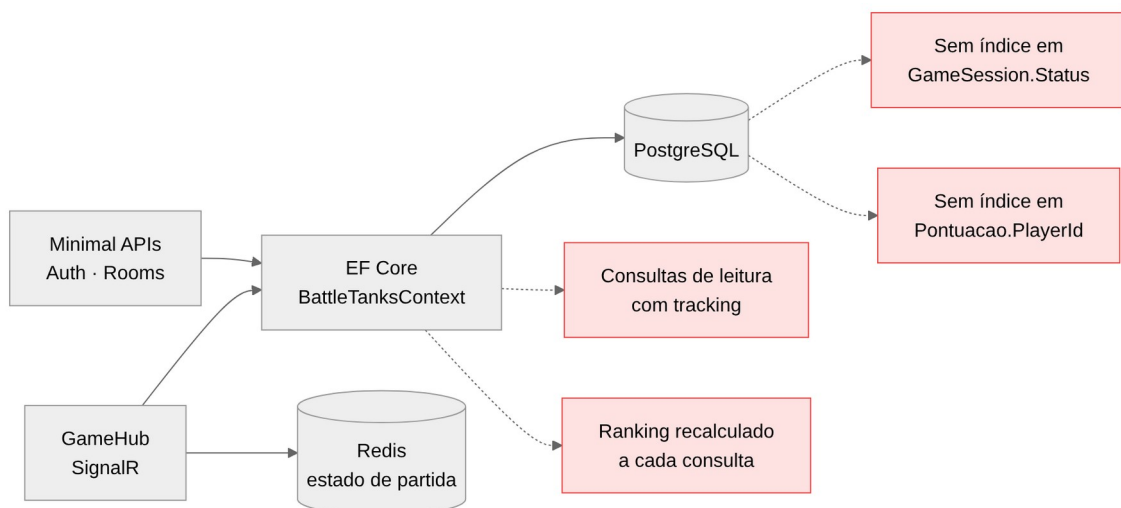
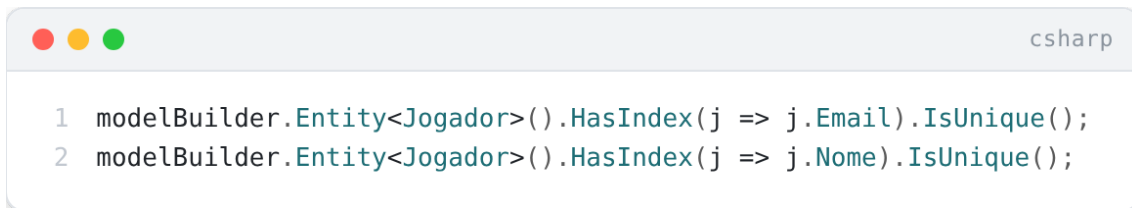


Figura 1 — Onde estão os custos hoje

O `OnModelCreating` atual declara apenas dois índices, ambos de unicidade em `Jogador`:



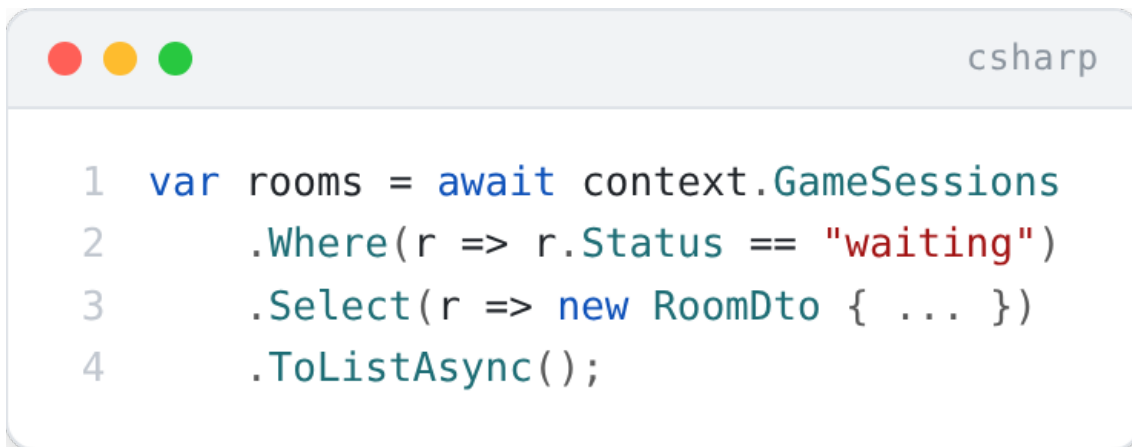
```
1 modelBuilder.Entity<Jogador>().HasIndex(j => j.Email).IsUnique();
2 modelBuilder.Entity<Jogador>().HasIndex(j => j.Nome).IsUnique();
```

São índices de **integridade**, não de desempenho: existem para impedir e-mail duplicado, não para acelerar consulta. As consultas que realmente rodam com frequência não têm suporte de índice nenhum.

3. Atividade #1 — Otimização de consultas com PostgreSQL e EF Core

3.1. Criação de índices

A consulta mais frequente da API é a listagem de salas disponíveis, em `RoomEndpoints.cs`:



```
1 var rooms = await context.GameSessions
2     .Where(r => r.Status == "waiting")
3     .Select(r => new RoomDto { ... })
4     .ToListAsync();
```

Sem índice em `Status`, o PostgreSQL faz *sequential scan*: lê a tabela inteira e descarta o que não interessa. Com poucas salas isso é irrelevante; com o histórico de partidas acumulando ao longo do semestre, o custo cresce linearmente enquanto o resultado útil permanece pequeno — o pior formato de consulta possível.

```
csharp
1 protected override void OnModelCreating(ModelBuilder modelBuilder)
2 {
3     base.OnModelCreating(modelBuilder);
4
5     modelBuilder.Entity<Jogador>().HasIndex(j => j.Email).IsUnique();
6     modelBuilder.Entity<Jogador>().HasIndex(j => j.Nome).IsUnique();
7
8     // Lobby: filtra salas por status a cada carregamento da tela.
9     modelBuilder.Entity<GameSession>()
10         .HasIndex(g => g.Status)
11         .HasDatabaseName("IX_GameSession_Status");
12
13     // Historico de pontuacao por jogador, ja ordenado pela data.
14     // Indice composto: o PostgreSQL usa o mesmo indice para filtrar E ordenar,
15     // eliminando o passo de sort.
16     modelBuilder.Entity<Pontuacao>()
17         .HasIndex(p => new { p.PlayerId, p.DataRegistro })
18         .HasDatabaseName("IX_Pontuacao_Player_Data");
19
20     // Ranking global: ordenacao decrescente por pontuacao acumulada.
21     modelBuilder.Entity<Jogador>()
22         .HasIndex(j => j.PontuacaoTotal)
23         .IsDescending()
24         .HasDatabaseName("IX_Jogador_PontuacaoTotal_Desc");
25 }
```

A ordem das colunas no índice composto não é acidental. (PlayerId, DataRegistro) serve para "as pontuações deste jogador, das mais recentes para as antigas". O inverso, (DataRegistro, PlayerId), não serviria: um índice só é aproveitado da esquerda para a direita, e a consulta filtra por jogador antes de ordenar por data.

Como confirmar que o índice está sendo usado:

```
sql
1 EXPLAIN ANALYZE
2 SELECT * FROM "GameSessions" WHERE "Status" = 'waiting';
```

Antes do índice, o plano mostra `Seq Scan on GameSessions`. Depois, deve mostrar `Index Scan using IX_GameSession_Status`. Se continuar em `Seq Scan` com a tabela pequena, está correto: o planejador ignora o índice quando ler tudo é mais barato que consultar o índice — o ganho aparece com volume.

3.2. Consultas somente leitura com `AsNoTracking()`

Por padrão o EF Core cria um *snapshot* de cada entidade retornada, para detectar alterações no `SaveChanges()`. Numa consulta que só exibe dados, esse trabalho é integralmente desperdiçado — custa memória e CPU para rastrear objetos que ninguém vai modificar.

```
csharp
1 group.MapGet("/", async (BattleTanksContext context) =>
2 {
3     var rooms = await context.GameSessions
4         .AsNoTracking() // nada aqui sera alterado
5         .Where(r => r.Status == "waiting")
6         .Select(r => new RoomDto
7         {
8             Id = r.Id,
9             Nome = r.Nome,
10            JogadoresConectados = r.JogadoresConectados,
11            MapaSelecioneado = r.MapaSelecioneado,
12            CapacidadeMaxima = r.CapacidadeMaxima,
13            Status = r.Status
14        })
15        .ToListAsync();
16
17    return Results.Ok(rooms);
18 });
```

Vale registrar uma nuance que muita gente erra: quando a consulta já projeta para um DTO com `Select`, o EF Core **não rastreia** o resultado, porque `RoomDto` não é entidade. Nesse caso específico o `AsNoTracking()` é redundante. Ele foi mantido por dois motivos: declara a intenção de leitura para quem lê o código, e protege a consulta caso alguém remova a projeção no futuro. Onde ele faz diferença real é em consultas que retornam a entidade:

```
csharp
1 // Aqui o ganho é concreto: retorna Jogador, entidade rastreavel.
2 var ranking = await context.Jogadores
3     .AsNoTracking()
4     .OrderByDescending(j => j.PontuacaoTotal)
5     .Take(10)
6     .ToListAsync();
```

3.3. Paginação

`ToListAsync()` sem limite traz a tabela inteira para a memória do processo. Numa

tela de ranking, isso é carregar milhares de linhas para exibir dez.

```
csharp
1 group.MapGet("/ranking", async (BattleTanksContext context, int pagina = 1, int tamanho = 20) =>
2 {
3     // Teto no tamanho da pagina: sem ele, ?tamanho=1000000 vira negacao de servico.
4     tamanho = Math.Clamp(tamanho, 1, 100);
5     pagina = Math.Max(pagina, 1);
6
7     var total = await context.Jogadores.CountAsync();
8
9     var itens = await context.Jogadores
10         .AsNoTracking()
11         .OrderByDescending(j => j.PontuacaoTotal)
12         .ThenBy(j => j.Id) // desempate estavel
13         .Skip((pagina - 1) * tamanho)
14         .Take(tamanho)
15         .Select(j => new { j.Id, j.Nome, j.PontuacaoTotal, j.Vitorias, j.JogosDisputados })
16         .ToListAsync();
17
18     return Results.Ok(new { total, pagina, tamanho, itens });
19 });
```

O `ThenBy(j => j.Id)` resolve um bug sutil: sem critério de desempate, dois jogadores com a mesma pontuação podem trocar de posição entre requisições, fazendo um deles aparecer duas vezes e outro sumir ao paginar. Ordenação de paginação precisa ser **total**, não parcial.

3.4. Operações em massa

Atualizar mil registros com o padrão "carregar, alterar, salvar" gera mil **UPDATE** individuais, além do custo de materializar todas as entidades. O EF Core 7+ resolve isso com `ExecuteUpdateAsync` e `ExecuteDeleteAsync`, que traduzem para um único comando SQL sem carregar nada para a memória.

```
csharp
1 // Encerra em lote as salas abandonadas: um UPDATE, nenhuma entidade carregada.
2 var encerradas = await context.GameSessions
3     .Where(g => g.Status == "in_game" && g.JogadoresConectados == 0)
4     .ExecuteUpdateAsync(s => s.SetProperty(g => g.Status, "finished"));
5
6 // Limpeza do historico antigo: um DELETE.
7 var limite = DateTime.UtcNow.AddMonths(-6);
8 var removidas = await context.Pontuacoes
9     .Where(p => p.DataRegistro < limite)
10     .ExecuteDeleteAsync();
```

Há um porém que precisa ser dito: essas operações **não passam pelo change tracker**. Se houver entidade carregada em memória, ela fica desatualizada, e nenhum evento de domínio é disparado. São a ferramenta certa para

manutenção em lote, e a errada para regra de negócio que dependa de eventos.

4. Atividade #2 — Integração Redis para cache distribuído

4.1. O que o Redis já faz, e o que falta

O `RedisStateService` da LB-6 guarda estado **efêmero de partida**: paredes destruídas num hash e os últimos 100 eventos numa lista, com `LPUSH` + `LTRIM`. Falta a ele o segundo papel que o laboratório pede: **cache de dados caros e gerenciamento de sessão**.

4.2. Cache do ranking global

O ranking é o candidato óbvio: caro de calcular, consultado com frequência e tolerante a alguns segundos de defasagem.

```
1 public class RankingCacheService
2 {
3     private const string CHAVE = "ranking:global";
4     private static readonly TimeSpan TTL = TimeSpan.FromMinutes(2);
5
6     private readonly IDatabase _db;
7     private readonly BattleTanksContext _ctx;
8
9     public async Task<List<RankingDto>> ObterTop10Async()
10    {
11        var cacheado = await _db.StringGetAsync(CHAVE);
12        if (cacheado.HasValue)
13            return JsonSerializer.Deserialize<List<RankingDto>>(cacheado!);
14
15        var ranking = await _ctx.Jogadores
16            .AsNoTracking()
17            .OrderByDescending(j => j.PontuacaoTotal)
18            .ThenBy(j => j.Id)
19            .Take(10)
20            .Select(j => new RankingDto(j.Id, j.Nome, j.PontuacaoTotal, j.Vitorias))
21            .ToListAsync();
22
23        // TTL curto: ranking desatualizado por 2 min e aceitavel;
24        // cache que nunca expira e bug esperando acontecer.
25        await _db.StringSetAsync(CHAVE, JsonSerializer.Serialize(ranking), TTL);
26        return ranking;
27    }
28
29    /// Invalida ao fim da partida, para o jogador ver seu resultado refletido.
30    public Task InvalidarAsync() => _db.KeyDeleteAsync(CHAVE);
31 }
```

A estratégia é **cache-aside com TTL curto e invalidação explícita**. O TTL é a rede de segurança para o caso de a invalidação falhar; a invalidação é o que dá a sensação de resposta imediata quando a partida termina. Confiar apenas em um dos dois deixa o sistema ou lento para atualizar, ou permanentemente inconsistente.

4.3. Ranking com Sorted Set — a alternativa melhor

Serializar JSON no Redis funciona, mas desperdiça a estrutura de dados que resolve exatamente este problema. Um **Sorted Set** mantém o ranking ordenado no próprio Redis:



```
1 // Ao fim de cada partida: atualiza a pontuacao do jogador no ranking.
2 await _db.SortedSetAddAsync("ranking:zset", jogadorId.ToString(), pontuacaoTotal);
3
4 // Top 10, ja ordenado, sem tocar no PostgreSQL e sem ordenar nada em C#.
5 var top = await _db.SortedSetRangeByRankWithScoresAsync(
6     "ranking:zset", 0, 9, Order.Descending);
7
8 // Posicao de um jogador especifico - operacao O(log N).
9 var posicao = await _db.SortedSetRankAsync(
10     "ranking:zset", jogadorId.ToString(), Order.Descending);
```

A última linha é o argumento decisivo. Responder "em que posição eu estou?" com SQL exige contar quantos jogadores têm pontuação maior — uma varredura. Com Sorted Set é $O(\log N)$, e a resposta é imediata mesmo com muitos jogadores.

4.4. Gerenciamento de sessão

Guardar sessão em memória do processo impede escalar horizontalmente: com duas instâncias atrás de um balanceador, o jogador autenticado numa perde a sessão ao cair na outra. O Redis resolve por ser estado compartilhado e externo.


```

1 public async Task RegistrarSessaoAsync(string sessaoId, int jogadorId, string salaId)
2 {
3     var chave = $"sessao:{sessaoId}";
4     await _db.HashSetAsync(chave, new[]
5     {
6         new HashEntry("jogadorId", jogadorId),
7         new HashEntry("salaId", salaId),
8         new HashEntry("desde", DateTimeOffset.UtcNow.ToUnixTimeSeconds())
9     });
10
11     // Expiracao obrigatoria: sem TTL, sessao de quem fechou o navegador
12     // fica para sempre e o Redis vira um vazamento de memoria lento.
13     await _db.KeyExpireAsync(chave, TimeSpan.FromHours(4));
14 }
15
16 public Task RenovarAsync(string sessaoId) =>
17     _db.KeyExpireAsync($"sessao:{sessaoId}", TimeSpan.FromHours(4));

```

4.5. Resumo das estruturas

Chave	Estrutura	Uso	Expiração
room:{id}:walls	Hash	Paredes destruídas (já existe)	Ao encerrar a sala
room:{id}:events	List (LTRIM 0-99)	Últimos eventos (já existe)	Ao encerrar a sala
ranking:zset	Sorted Set	Ranking global ordenado	Permanente
ranking:global	String (JSON)	Cache do top 10	2 min
sessao:{id}	Hash	Sessão do jogador	4 h, renovável

4.6. Jogadores conectados com Set

O enunciado pede o armazenamento dos jogadores conectados a partir do GameHub. A estrutura correta é o **Set**, porque a propriedade que interessa é unicidade: o mesmo jogador não pode constar duas vezes, e a contagem precisa ser barata.

```

1 // Em GameHub.JoinRoom, apos Groups.AddToGroupAsync:
2 await _db.SetAddAsync($"sala:{payload.RoomId}:online", payload.PlayerId);
3 await _db.SetAddAsync("jogadores:online", payload.PlayerId);
4
5 // Em LeaveRoom e em OnDisconnectedAsync:
6 await _db.SetRemoveAsync($"sala:{payload.RoomId}:online", payload.PlayerId);
7
8 // Contagem em O(1), sem varrer o conjunto:
9 var online = await _db.SetLengthAsync("jogadores:online");

```

SCARD responde a contagem em tempo constante, e SADD é idempotente — chamar duas vezes com o mesmo jogador não duplica. Isso importa porque uma reconexão rápida pode disparar `JoinRoom` antes de o `OnDisconnectedAsync` anterior ter rodado.

4.7. Sessão e revogação de token

O enunciado pede validar a sessão **sem consultar o PostgreSQL**, e guardar tokens JWT expirados no Redis. O ponto sutil é que um JWT é autocontido e válido até expirar — não existe "apagar" um JWT. A forma de revogá-lo antes do prazo é manter uma **lista de negação** consultada a cada requisição.

```

1 /// Revoga um token no logout. O TTL e o tempo que faltava para ele expirar
2 /// sozinho: depois disso a negacao e desnecessaria, e a chave some por conta
3 /// propria - a lista nunca cresce indefinidamente.
4 public async Task RevogarTokenAsync(string jti, DateTimeOffset expiraEm)
5 {
6     var restante = expiraEm - DateTimeOffset.UtcNow;
7     if (restante <= TimeSpan.Zero) return;
8
9     await _db.StringSetAsync($"jwt:revogado:{jti}", "1", restante);
10 }
11
12 public Task<bool> TokenRevogadoAsync(string jti) =>
13     _db.KeyExistsAsync($"jwt:revogado:{jti}");

```

Casar o TTL da chave com o tempo restante do token é a decisão que torna isso sustentável. Sem ela, a lista de negação cresceria para sempre guardando tokens que já expiraram e não poderiam ser usados de qualquer forma.

5. Atividade #3 — Design do banco para escalabilidade

5.1. Sharding por região

O enunciado sugere fragmentar por região, e faz sentido no contexto de um jogo: a partida é uma interação **local entre poucos jogadores**, e ninguém joga com quem está do outro lado do mundo por causa da latência. Isso significa que a fronteira natural de fragmentação já existe no domínio.

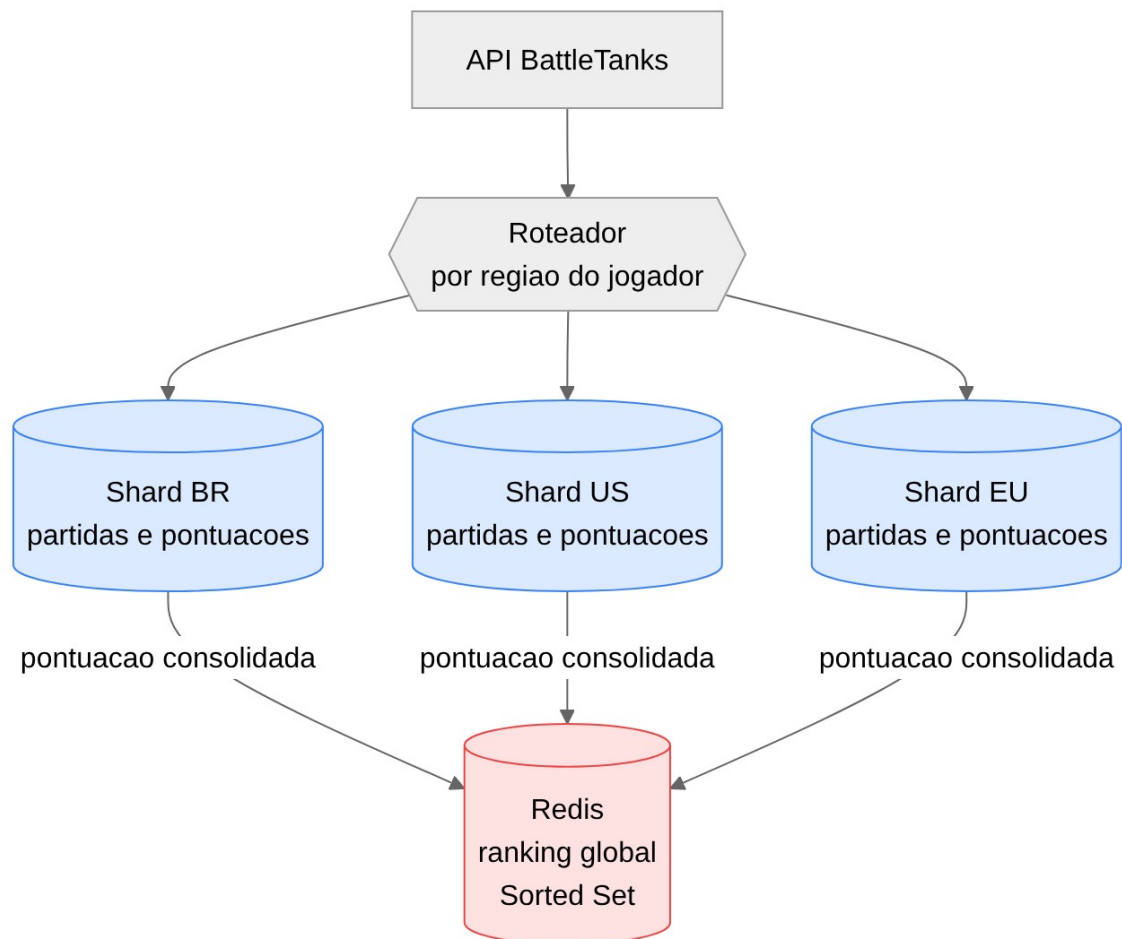


Figura 2 — Sharding por região, com ranking centralizado no Redis

A chave de fragmentação seria a região do jogador, e o dado de partida ficaria no shard onde a partida aconteceu. Duas consequências precisam ser declaradas:

O que fica fácil. Consultas de partida, histórico e estatística de um jogador ficam inteiramente dentro de um shard. Nenhuma consulta distribuída, nenhuma transação distribuída.

O que quebra. O ranking global. Ordenar todos os jogadores exigiria consultar cada shard, trazer resultados parciais e mesclar — e o resultado seria uma foto de instantes diferentes, com paginação incorreta. A solução adotada no desenho acima é **não fazer ranking global no banco**: cada shard publica a pontuação consolidada num Sorted Set do Redis, que é central e ordenado por natureza.

Vale a honestidade sobre o escopo: o **Battle Tanks não precisa de sharding hoje**. Alguns milhares de jogadores não saturam uma instância PostgreSQL bem indexada. O que o projeto precisa de fato é do particionamento por data, discutido na tarefa 7.4, que ataca o problema real — crescimento do histórico. Sharding entra no relatório como projeto para escala futura, como o enunciado pede, e não como necessidade atual.

5.2. Réplicas de leitura com replicação em streaming

Réplicas resolvem um problema mais imediato que o sharding: distribuir a **carga de leitura** sem multiplicar a de escrita.



Configuração no primário (postgresql.conf):

```
conf

1 wal_level = replica
2 max_wal_senders = 4
3 wal_keep_size = 512MB
4 hot_standby = on
```

E a réplica é criada a partir de um `pg_basebackup`, seguindo o WAL do primário continuamente.

No lado da aplicação, a separação é por string de conexão:

```
1 builder.Services.AddDbContext<BattleTanksContext>(o =>
2     o.UseNpgsql(cfg.GetConnectionString("Primario")));
3
4 // Contexto somente leitura, apontado para a replica.
5 builder.Services.AddDbContext<BattleTanksReadContext>(o =>
6     o.UseNpgsql(cfg.GetConnectionString("Replica"))
7     .UseQueryTrackingBehavior(QueryTrackingBehavior.NoTracking));
```

NoTracking fixado no contexto de leitura não é só otimização: é uma **declaração de intenção**. Ele não bloqueia a escrita por si — quem chamar Add e SaveChanges ainda tenta gravar — mas remove o caminho acidental, o de carregar uma entidade, alterá-la e salvar sem perceber. A barreira de verdade é a própria réplica, que é somente leitura e devolve erro em tempo de execução.

O trade-off a declarar é o **atraso de replicação**. A réplica fica milissegundos ou segundos atrás do primário. Para ranking e histórico isso é irrelevante. Para a tela que o jogador vê logo após terminar a partida, não é — ali a leitura precisa ir ao primário, ou ele acha que a pontuação não foi contada.

5.3. Connection pooling com Npgsql

Abrir conexão com PostgreSQL é caro: envolve handshake TCP, autenticação e alocação de um processo no servidor. Num jogo com muitas requisições curtas, o custo de abrir a conexão supera o da consulta.

O Npgsql mantém pool por padrão, mas os valores default raramente servem:

```
Host=localhost;Database=battletanks;Username=app;Password=***;
Minimum Pool Size=5;
Maximum Pool Size=50;
Connection Idle Lifetime=300;
Timeout=15;
Command Timeout=30;
Max Auto Prepare=20
```

Parâmetro	Por quê
Minimum Pool Size=5	Evita o custo de abrir conexão no primeiro acesso após ociosidade
Maximum Pool Size=50	Teto. Precisa ser menor que o <code>max_connections</code> do servidor dividido pelo número de instâncias da API
Connection Idle Lifetime	Devolve ao servidor conexões paradas

Timeout	Falha rápido quando o pool esgota, em vez de acumular requisições esperando
Max Auto Prepare	Prepara automaticamente as consultas mais repetidas, economizando o replanejamento

O erro clássico aqui é **dimensionar o pool grande demais**. Cada conexão consome um processo no PostgreSQL; um pool de 200 conexões por instância, com três instâncias, esgota o `max_connections` padrão e derruba o banco. Pool menor com fila costuma dar mais vazão do que pool grande com o servidor sobrecarregado. Acima de certo ponto, um *pooler* externo como o PgBouncer é o caminho.

5.4. Benchmarking de carga

O enunciado marca esta parte como **opcional**, e ela ficou fora do escopo desta entrega. Fica registrado o caminho para quando for feita: simular mais de 100 conexões simultâneas com k6 ou JMeter, medir o tempo de resposta do `/api/rooms` e do ranking sob carga crescente, e plotar carga contra tempo de resposta para localizar o joelho da curva.

6. Como validar os ganhos

O que medir	Como
Uso de índice	<code>EXPLAIN ANALYZE</code> antes e depois — procurar <code>Index Scan</code> no lugar de <code>Seq Scan</code>
Consultas geradas pelo EF	Ativar <code>LogTo(Console.WriteLine)</code> e conferir o SQL emitido
Consulta N+1	Contar comandos no log: uma tela que emite dezenas de <code>SELECT</code> tem N+1
Latência do endpoint	<code>curl -w "%{time_total}"</code> no <code>/api/rooms</code> e no <code>/api/ranking</code>
Acerto do cache	<code>INFO stats</code> no Redis: <code>keyspace_hits</code> contra <code>keyspace_misses</code>

6.1. Medições feitas com PostgreSQL e Redis reais

A migration `AddPerformanceIndexes` foi gerada com `dotnet ef migrations add` e aplicada com `dotnet ef database update`, ambos sem erro. As medições abaixo rodaram sobre essa migration, com `GameSessions` populada com 50 mil linhas e `Jogadores` com 20 mil, para que o volume seja grande o bastante para o planejador do PostgreSQL preferir o índice — com poucas linhas ele prefere *seq scan*, como já registrado na seção 3.1.

Índice de ``GameSession.Status``. `EXPLAIN ANALYZE` na mesma consulta, com e sem o índice disponível:

```
Sem indice: Seq Scan on "GameSessions" — Execution Time: 4.895 ms (49900 linhas descartadas)
Com indice: Index Scan using IX_GameSession_Status — Execution Time: 0.482 ms
```

Confirma a previsão da seção 3.1: troca de `Seq Scan` por `Index Scan`, ~10x mais rápido.

``AsNoTracking()`` na consulta de ranking. Carregando as 20 mil linhas de `Jogadores` (cenário ampliado para tornar o custo do change tracker visível — o endpoint real usa `Take(10)` ou `Take(20)`, onde a diferença absoluta é pequena demais para medir):

```
Com tracking (padrao): 247 ms | 18.040 KB alocados
Com AsNoTracking(): 121 ms | 9.097 KB alocados
```

Aproximadamente metade do tempo e da memória, confirmando a seção 3.2.

Cache do ranking (Redis). `ObterTop10Async()` chamado duas vezes seguidas:

```
1a chamada (reconstroi do banco): 797 ms
2a chamada (cache hit): 5 ms
TTL da chave logo apos gravar: 119,988 s (config: 120 s)
```

Sorted Set e revogação de JWT. Testados com três jogadores de pontuações distintas: a posição via `ZREVRANK` bateu com a ordem esperada (1ª, 2ª, 3ª) e um jogador sem pontuação retornou posição nula, como a seção 4.3 previa. A revogação de token (seção 4.7) foi testada com um TTL curto de 3s: o token aparece como revogado imediatamente após `RevogarTokenAsync` e deixa de aparecer como revogado assim que o TTL expira — a chave some sozinha, sem acumular. Um token já expirado no momento da revogação não chega a gravar chave no Redis.

Paginação. Consultada a página 1 (OFFSET 0) e a página 2 (OFFSET 20) da mesma ordenação PontuacaoTotal DESC, Id: interseção de Id entre as duas páginas é zero, ou seja, o desempate por Id da seção 3.3 realmente elimina a duplicação/omissão de jogadores.

6.2. Bugs encontrados ao testar de verdade (e já corrigidos na branch)

Testar contra Postgres e Redis reais — em vez de só ler o código — expôs dois problemas que a leitura não pegaria:

A réplica de leitura nunca subia. O `docker-compose.yml` da seção 5.2 tinha três defeitos empilhados, todos silenciosos até alguém rodar `docker compose up` de verdade:

1. O bloco `command: > bash -c "..."` é um *scalar* YAML dobrado; a linha de continuação do

`pg_basebackup (-D /var/lib/postgresql/data ...)` ficava indentada demais e o YAML a preservava como uma linha separada, virando um comando `-D` inexistente no bash.

2. O `pg_hba.conf` gerado pelo `initdb` libera `host all all all`, mas isso **não cobre** a

`pseudo-database replication` — faltava uma entrada dedicada, e a réplica era rejeitada com `no pg_hba.conf entry for replication connection`.

3. Depois de corrigir os dois itens acima, o processo final (`exec postgres ...`) ainda falhava

com `"root" execution ... is not permitted`: o `bash -c` do `command`: roda como `root` e pula a troca de usuário que o entrypoint oficial da imagem faz sozinho — precisa de `exec gosu postgres postgres ...` explícito. E como o script inteiro passou a rodar como `root`, os arquivos do `pg_basebackup` ficavam com dono `root`; faltava um `chown -R postgres` antes do `gosu` assumir.

Com as três correções, a réplica sobe, `pg_stat_replication` no primário mostra `state = streaming`, um `INSERT` no primário aparece na réplica em menos de um segundo, e um `INSERT` direto na réplica falha com `cannot execute INSERT in a read-only transaction` — o comportamento que a seção 5.2 descrevia, agora confirmado rodando.

``jogadores:online`` só encolhia com desconexão educada. O `GameHub` (seção 4.6) nunca sobrescrevia `OnDisconnectedAsync`: o `Set` só perdia um jogador

quando o cliente chamava `LeaveRoom` explicitamente. Queda de conexão, aba fechada ou crash do app — o caminho mais comum de desconexão numa partida — deixavam o jogador para sempre no Set, inflando `SCARD` com o tempo. A correção mapeia `ConnectionId` → (`RoomId`, `PlayerId`) no `JoinRoom` e usa esse mapa em `OnDisconnectedAsync` para repetir a mesma limpeza que `LeaveRoom` já fazia.

Um terceiro ponto ficou identificado mas **não corrigido**, por estar fora do escopo do que foi testado agora: `RevogarTokenAsync/TokenRevogadoAsync` (seção 4.7) funcionam corretamente isolados, mas nenhum endpoint de logout chama `RevogarTokenAsync`, e nada no pipeline de autenticação consulta `TokenRevogadoAsync` — hoje a revogação existe como código morto, sem efeito sobre requisições reais.

7. Conclusões sobre o progresso do capstone

O código desta análise está aplicado na branch `lab7-djordan` do repositório do grupo.

O laboratório 6 resolveu a comunicação; este ataca o que a sustenta. São problemas de natureza diferente: lá o gargalo era latência de entrega, aqui é custo de consulta — e custo de consulta só aparece com volume, o que o torna invisível em desenvolvimento e visível em apresentação.

Três lições ficam registradas para o projeto final. A primeira é que **índice não é otimização genérica**: cada um dos quatro propostos existe por causa de uma consulta específica que roda no código, e índice sem consulta correspondente só encarece a escrita. A segunda é que **cache exige política de invalidação desde o primeiro dia** — TTL sozinho produz dados velhos, invalidação sozinha falha em silêncio. A terceira é que **a escolha da estrutura de dados vale mais que a escolha da tecnologia**: trocar JSON serializado por Sorted Set no mesmo Redis transforma a consulta de posição no ranking de varredura em $O(\log N)$.

Uma quarta lição veio de fora da análise de banco: **rodar é diferente de ler**. O `docker-compose.yml` da réplica de leitura parecia correto na leitura — três defeitos diferentes (YAML, `pg_hba.conf`, usuário do processo) só apareceram ao executar `docker compose up` de verdade, e o mesmo vale para o `OnDisconnectedAsync` que faltava no `GameHub`. Os ganhos de índice e `AsNoTracking()` deixaram de ser previsão fundamentada: a seção 6.1 traz os números medidos com `EXPLAIN ANALYZE` e um comparativo de

tempo/alocação sobre PostgreSQL e Redis reais, e a seção 6.2 documenta os dois bugs que a execução expôs e que já foram corrigidos na branch `lab7-djordan`.

8. Referências

- Microsoft. *EF Core — Índices*.
<<https://learn.microsoft.com/ef/core/modeling/indexes>>
- Microsoft. *EF Core — Tracking vs. No-Tracking Queries*.
<<https://learn.microsoft.com/ef/core/querying/tracking>>
- Microsoft. *EF Core — ExecuteUpdate e ExecuteDelete*.
<<https://learn.microsoft.com/ef/core/saving/execute-insert-update-delete>>
- PostgreSQL. *Using EXPLAIN*.
<<https://www.postgresql.org/docs/current/using-explain.html>>
- Redis. *Sorted Sets*. <<https://redis.io/docs/latest/develop/data-types/sorted-sets/>>
- Redis. *Client-side caching e estratégias de cache*.
<<https://redis.io/docs/latest/develop/reference/client-side-caching/>>
- Repositório do grupo, branch `LB-6` — esquema e `RedisStateService` de partida.